FluxOS - API Billing Platform Documentation

Introduction

FluxOS - API Billing Platform is a full-stack system designed to manage, monitor, and monetize API usage in a scalable and secure way.

The platform allows developers to:

1. Generate secure API Keys
2. Access protected gateway endpoints
3. Track API usage in real-time
4. Enforce rate limits using redis
5. Upgrade subscription plans (Free → Pro)
6. Visualize analytics via a React dashboard

This project simulates a real-world **API monetization platform**, similar to systems used by companies like OpenAI, Stripe, and OpenWeather API.

Design Inspiration

This project is inspired by real-world API platform but implements a custom system:

1. **Pricing Model Inspiration:** Inspired by OpenWeather API pricing tiers, where users get different usage limits based on subscription plans(FREE / PRO).
2. **API Response Inspiration :** Inspired by PokeAPI, which returns dynamic and varied responses for each request.

3. **<u>FluxOS Custom Implementation :</u>** Instead of returning domain-specific data, FluxOS generates **random Operating System information** on each API call, including:

- Linux distributions
- Windows Versions
- MAcOS Versions

This makes the System Dynamic, Engaging and Ideal for demonstration and testing API behavior.

<u>Roles</u>

1. <u>Users</u>
    - Registers and logs in
    - Generates API keys
    - Calls gateway APIs
    - Views usage analytics
    - Upgrades subscription plan

2. <u>Admin (Implicit Role)</u>
    - Defines plans (Free / Pro)
    - Controls pricing and quotas
    - Manages system behavior

<u>Use Cases of the Project</u>

1. <u>User SignIn / Login</u>
    - Generates API Keys
    - Calls Gateway API (/gateway/data)
    - System validates request
    - Applies rate limiting (Redis)

- Track usage in database
- Returns dynamic OS data
- Dashboard shows analytics
- User upgrades plan if needed

Key Use Cases

1. Secure API access using API Keys
2. Rate limiting with Redis
3. Usage tracking and analytics
4. Plan-based billing system
5. Plan upgrade functionality
6. API playground with dynamic response

Tech Stack

1. **Backend**
   - **SpringBoot**
     1. Rapid REST API development
     2. Clean Layered architecture (Controller → Service→ Repository)
   - **Spring Security**
     1. Secures endpoints
     2. Integrates with JWT authentication
   - **JWT (Json Web Token)**
     1. Stateless authentication
     2. Scalable and secure
   - **PostgreSQL**
     1. Relational database
     2. Stores users, API keys, Usage data
   - **Redis**
     1. High Speed in-memory store
     2. Used for rate limiting counters

2. **<u>Frontend</u>**
   - **React**
     1. Component-based UI
     2. Efficient state management
     3. Scalable frontend architecture
   - **TailWind CSS**
     1. Utility-first styling
     2. Clean and modern UI
     3. Faster development workflow

<u>Authentication</u>

Authentication is implemented using JWT(Json Web Token).

Flow:

1. User send login request :
   POST /auth/login
2. Backend validates credentials
3. JWT token is generated
   Authorization: Bearer <JWT_TOKEN>
4. Token is used for all protected APIs

Security Layers:

1. JwtFilter : Validates JWT for protected routes
2. ApiKeyFilter
   - Validates API Key
   - Applies rate limiting using Redis
   - Tracks API usage in database

- Allows or blocks requests

Flowcharts

🔹 Backend Flow

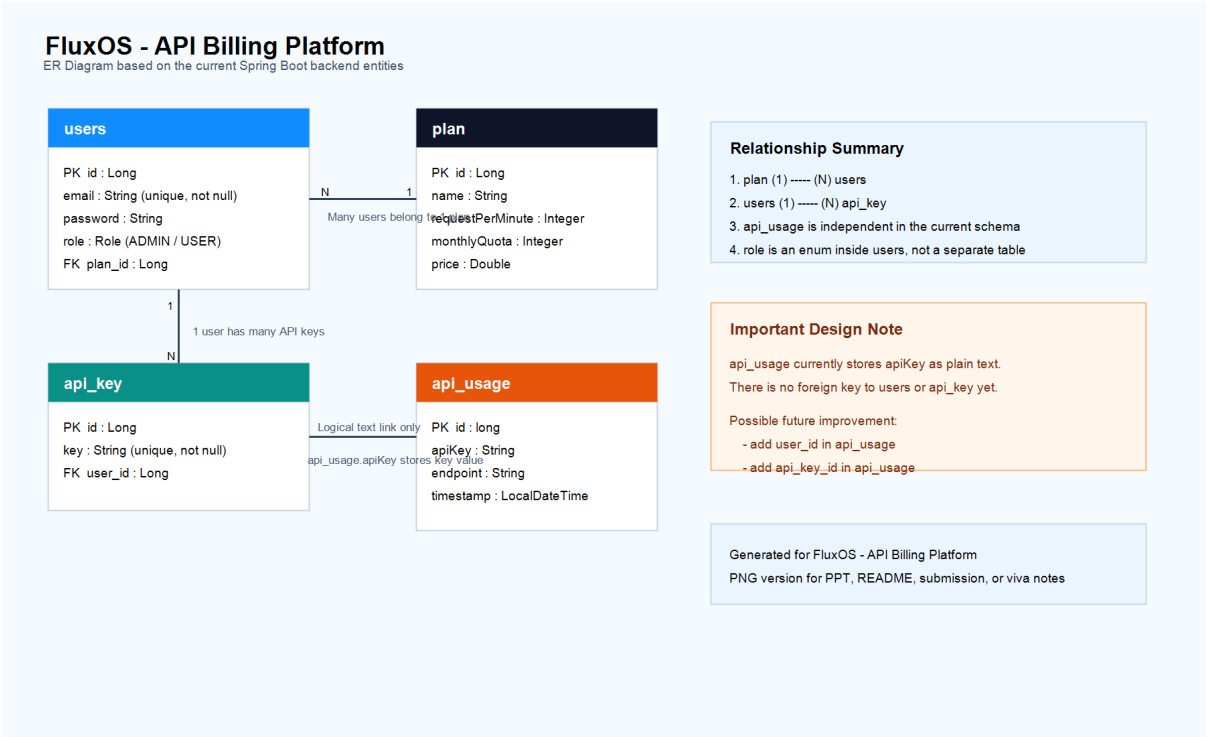
Client
→ JWT Authentication
→ API Key Generation
→ Gateway Request
→ ApiKeyFilter
→ Redis Rate Limit Check
→ Usage Tracking
→ Response Returned

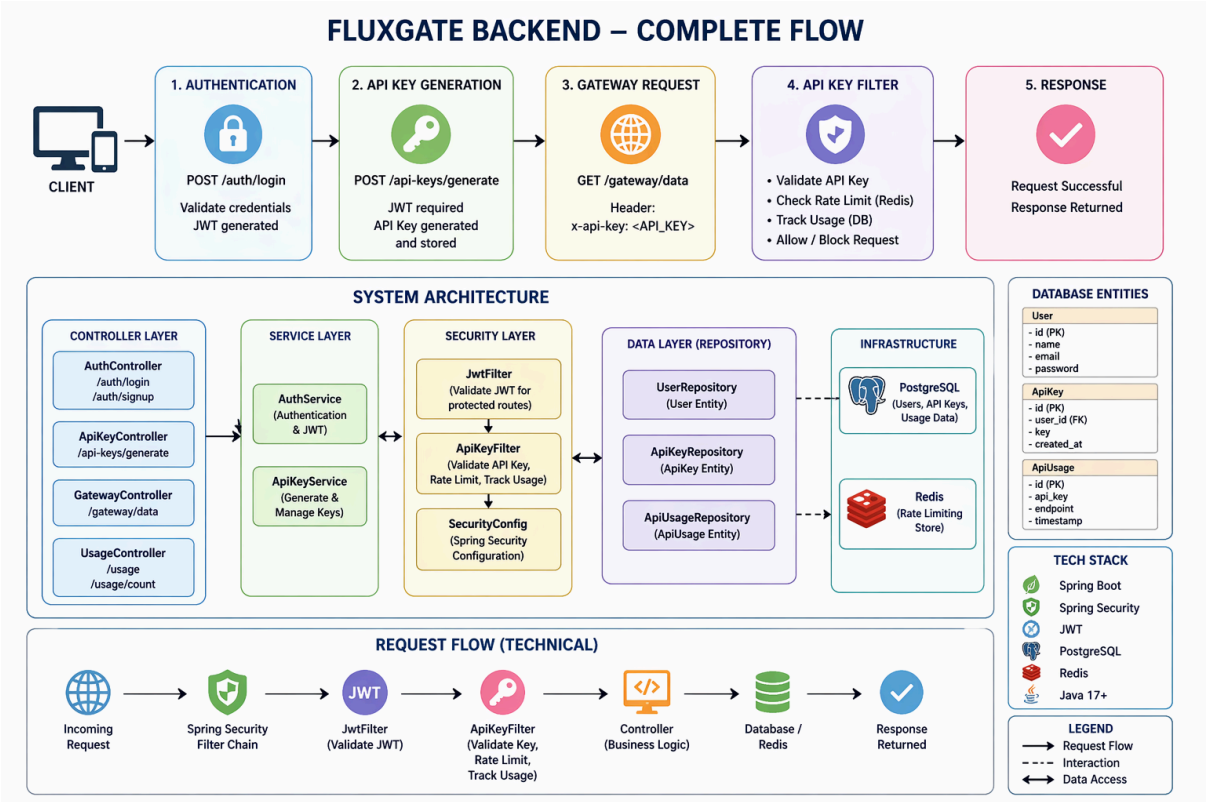
🔹 System Architecture

- Controller Layer → Handles API requests
- Service Layer → Business logic
- Security Layer → JWT + API Key filters
- Repository Layer → Database operations
- Infrastructure → PostgreSQL + Redis

🔹 Database Design (ER Concept)

- User → has many API Keys
- Plan → assigned to Users
- API Key → belongs to User
- API Usage → linked to API Key


FluxOS - API Billing Platform
ER Diagram based on the current Spring Boot backend entities
users
PK id : Long
email : String (unique, not null)
password : String
role : Role (ADMIN / USER)
FK plan_id : Long
N
1
Many users belong to one plan
plan
PK id : Long
name : String
requestPerMinute : Integer
monthlyQuota : Integer
price : Double
1
N
1 user has many API keys
api_key
PK id : Long
key : String (unique, not null)
FK user_id : Long
Logical text link only
api_usage.apiKey stores key value
api_usage
PK id : long
apiKey : String
endpoint : String
timestamp : LocalDateTime
Relationship Summary
1. plan (1) ----- (N) users
2. users (1) ----- (N) api_key
3. api_usage is independent in the current schema
4. role is an enum inside users, not a separate table
Important Design Note
api_usage currently stores apiKey as plain text.
There is no foreign key to users or api_key yet.
Possible future improvement:
- add user_id in api_usage
- add api_key_id in api_usage
Generated for FluxOS - API Billing Platform
PNG version for PPT, README, submission, or viva notes


FLUXGATE BACKEND – COMPLETE FLOW
CLIENT
1. AUTHENTICATION
POST /auth/login
Validate credentials
JWT generated
2. API KEY GENERATION
POST /api-keys/generate
JWT required
API Key generated
and stored
3. GATEWAY REQUEST
GET /gateway/data
Header:
x-api-key: <API_KEY>
4. API KEY FILTER
• Validate API Key
• Check Rate Limit (Redis)
• Track Usage (DB)
• Allow / Block Request
5. RESPONSE
Request Successful
Response Returned
SYSTEM ARCHITECTURE
CONTROLLER LAYER
AuthController
/auth/login
/auth/signup
ApiKeyController
/api-keys/generate
GatewayController
/gateway/data
UsageController
/usage
/usage/count
SERVICE LAYER
AuthService
(Authentication & JWT)
ApiKeyService
(Generate & Manage Keys)
SECURITY LAYER
JwtFilter
(Validate JWT for protected routes)
ApiKeyFilter
(Validate API Key, Rate Limit, Track Usage)
SecurityConfig
(Spring Security Configuration)
DATA LAYER (REPOSITORY)
UserRepository
(User Entity)
ApiKeyRepository
(ApiKey Entity)
ApiUsageRepository
(ApiUsage Entity)
INFRASTRUCTURE
PostgreSQL
(Users, API Keys, Usage Data)
Redis
(Rate Limiting Store)
DATABASE ENTITIES
User
- id (PK)
- name
- email
- password
ApiKey
- id (PK)
- user_id (FK)
- key
- created_at
ApiUsage
- id (PK)
- api_key
- endpoint
- timestamp
TECH STACK
Spring Boot
Spring Security
JWT
PostgreSQL
Redis
Java 17+
REQUEST FLOW (TECHNICAL)
Incoming Request
Spring Security Filter Chain
JWT
JwtFilter (Validate JWT)
ApiKeyFilter (Validate Key, Rate Limit, Track Usage)
Controller (Business Logic)
Database / Redis
Response Returned
LEGEND
Request Flow
Interaction
Data Access

Industry Value of the Project

This project demonstrates real-world backend engineering concepts:

Industry Applications

- API platforms (RapidAPI, OpenAI APIs)
- SaaS billing systems (Stripe-like)
- Developer platforms
- Cloud service APIs

Key Concepts Implemented

1. API Gateway Design
2. Rate Limiting using Redis
3. Usage Metering
4. Billing Systems
5. JWT Authentication
6. Full-stack Dashboard

Real-world Inspiration Applied

- OpenWeather → Usage-based pricing model
- PokeAPI → Dynamic API responses

FluxOS combines both ideas into a single system:

- Monetization logic
- Dynamic API behavior

Technologies Used

Backend Technologies

1. **Spring Boot**

    - Used to build RESTful APIs quickly and efficiently
    - Provides auto-configuration and embedded server support
    - Enables clean layered architecture (Controller → Service → Repository)
    - Simplifies dependency management and project setup

2. **Spring Security**

    - Handles authentication and authorization
    - Protects API endpoints using filters
    - Integrates seamlessly with JWT for stateless security
    - Provides filter chain for request validation

3. **JWT (JSON Web Token)**

    - Used for stateless authentication
    - Eliminates need for server-side session storage
    - Ensures secure communication between client and server
    - Passed in Authorization header for every protected request

4. **PostgreSQL**

- Relational database used for persistent storage
- Stores:
  - Users
  - API Keys
  - API Usage logs
  - Plans
- Provides strong data consistency and relational integrity
- Suitable for structured SaaS applications

5. **Redis**

- In-memory key-value store used for high-speed operations
- Implements rate limiting using request counters
- Stores temporary data like:
- API request counts
- Rate limit tracking per API key
- Ensures fast performance compared to traditional databases

6. **Java 17+**

- Core programming language for backend development
- Supports modern features like:
- Records
- Improved performance
- Better memory management

- Strong ecosystem and compatibility with Spring Boot

Frontend Technologies

1. **React**
    - Used to build dynamic and responsive user interfaces
    - Component-based architecture improves code reusability
    - Handles:
        - Dashboard rendering
        - API integration
        - State management using hooks(useState, useEffect)
        - Enables smooth user experience with real-time updates

2. **Tailwind CSS**
    - Utility-first CSS framework
    - Allows rapid UI development with predefined classes
    - Helps create:
        - Clean layouts
        - Responsive design
    - Modern SaaS-style dashboards
    - Reduces need for writing custom CSS

Integration Overview

React Frontend

↓ (HTTP Requests)

Spring Boot Backend

↓

PostgreSQL (Persistent Data)

+

Redis (Rate Limiting & Performance)

Conclusion

FluxOS (MeterFlow) successfully demonstrates the design and implementation of a scalable API billing platform.

Key achievements:

- Secure API access using JWT and API keys
- Rate limiting using Redis
- Usage tracking and analytics
- Plan-based billing system
- Full-stack dashboard integration

The project also draws inspiration from real-world APIs like OpenWeather for pricing models and PokeAPI for dynamic response design, while introducing a unique FluxOS concept that returns randomized operating system data for each request.

This makes the system both practical and engaging, and showcases strong backend, system design, and full-stack development skills.